# How to Run and Explain the C++ Internship Assignment

*Complete beginner-friendly guide **for WSL Ubuntu setup**, folder creation, pasting code, building, running, checking output, and explaining the workflow.*

Goal:

Part 1 extracts 640x640 JPEG frames from an MP4 video using GStreamer.

Part 2 reads those frames in C++, detects faces using an OpenCV pre-trained XML classifier, crops the detected faces, and stores them in frame-wise folders.

Before starting, keep these 3 things ready: (1) your input MP4 video, (2) the Part 1 shell script code, and (3) the Part 2 C++ code plus Makefile.

## 1. Final Project Structure

Place the files in a project folder with the following structure:

```
cpp_internship_assignment/
├── input.mp4          // Input video file used for testing
├── run_part1.sh       // Part 1 shell script for GStreamer pipeline
├── Makefile
├── face_cropper
├── src/
│   └── main.cpp    //Main source code  Part2

│
├── frames/          // Part 1 Output : Extracted JPEG frames

│
└── cropped_faces/   //Part 2 Output : cropped faces saved frame-wise
```

Notes:

- input.mp4 is the video used as input for Part 1.
- run_part1.sh contains the gst-launch pipeline.
- main.cpp contains the OpenCV-based face detection
- **frames/  is the Part 1 output**
- **cropped_faces/ is the Part 2 output folder.**

## 2. Step-by-Step: Create the Project Folder and Files

### 2.1 Open WSL Ubuntu

Start Ubuntu from WSL so all commands below run in Linux environment.

### 2.2 Go to your home folder

Run: cd ~

### 2.3 Create the main project folder

Run: mkdir -p ~/cpp_internship_assignment/src

### 2.4 Open the project folder

Run: cd ~/cpp_internship_assignment

### 2.5 Create the output folders

Run: mkdir -p frames cropped_faces

### 2.6 Place the input video

Copy your input MP4 into this folder and rename it to input.mp4.

### 2.7 Create the Part 1 script file

Run: nano run_part1.sh, then paste the shell script code, save with Ctrl+O, press Enter, and exit with Ctrl+X.

### 2.8 Create the Part 2 source file

Run: nano src/main.cpp, then paste the C++ code, save with Ctrl+O, press Enter, and exit with Ctrl+X.

### 2.9 Create the Makefile

Run: nano Makefile, then paste the Makefile content, save with Ctrl+O, press Enter, and exit with Ctrl+X.

## 3. Install Required Packages in WSL Ubuntu

Run the following commands one by one:

- `sudo apt update`
- `sudo apt install -y g++ make pkg-config unzip`
- `sudo apt install -y libopencv-dev`
- `sudo apt install -y gstreamer1.0-tools gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-libav`

Verify installation:

- `g++ --version`
- `gst-launch-1.0 --version`
- `pkg-config --modversion opencv4`

## 4. Get the OpenCV Haar Cascade XML File

The C++ program needs an OpenCV pre-trained XML classifier to detect faces. The recommended file is haarcascade_frontalface_alt2.xml.

- `find /usr -name "haarcascade_frontalface_alt2.xml" 2>/dev/null`
- `cp /usr/share/opencv4/haarcascades/haarcascade_frontalface_alt2.xml .`

If alt2 is not available, use the default classifier instead:

- `find /usr -name "haarcascade_frontalface_default.xml" 2>/dev/null`
- `cp /usr/share/opencv4/haarcascades/haarcascade_frontalface_default.xml .`

**Find the file name  and copy it into the main.cpp:**

## 5. What to Paste in Each File

### 5.1 run_part1.sh

Paste the GStreamer shell script into run_part1.sh.

This script reads input.mp4, decodes the video, resizes frames to 640x640, encodes them as JPEG images, and saves them in the frames folder.

### 5.2 src/main.cpp

Paste the Part 2 C++ program into src/main.cpp.

This file reads JPEG frames, loads the OpenCV XML classifier, detects faces, filters bad detections, crops the final face regions, and saves them in cropped_faces/frame_n folders.

### 5.3 Makefile

Paste the Makefile used to compile the C++ program with OpenCV. After running make, the compiled executable should be created as face_cropper.

## 6. Run Part 1: Extract JPEG Frames

Make the shell script executable and run it:

- `chmod +x run_part1.sh`
- `./run_part1.sh`

What this script does:

- filesrc reads the input MP4 file.
- qtdemux extracts the video stream from the MP4 container.
- decodebin decodes the compressed video into raw frames.
- videoconvert and videoscale prepare the frames and enable resizing.
- video/x-raw,width=640,height=640 forces the output resolution to 640x640.
- jpegenc converts each frame into JPEG format.
- multifilesink saves every frame as a separate JPEG image in the frames folder.

Check that Part 1 worked:

- `ls frames | head`
- `ls frames | wc -l`

**Output will be store in Frames Folder**

## 7. Build and Run Part 2: Detect and Crop Faces

Compile the C++ program using the Makefile:

- `make clean`
- `make`

Run the executable:

- `./face_cropper`

Check that Part 2 worked:

- `find cropped_faces -maxdepth 2 -type f | head -20`
- `find cropped_faces -maxdepth 2 -type f | wc -l`

# Output will be store in cropped_faces Folder

## 8. End-to-End Quick Command Flow

After creating the files and pasting the code, you can run everything in this order:

```
cd ~/cpp_internship_assignment
chmod +x run_part1.sh
./run_part1.sh
make clean
make
./face_cropper
ls frames | head
find cropped_faces -maxdepth 2 -type f | head -20
```

## 9. How the C++ Program Works

| Step | Why it is done |
|---|---|
| Read frame images | The program scans the frames folder, keeps only .jpg/.jpeg files, sorts them, and processes them one by one. |
| Load XML classifier | It loads OpenCV's pre-trained Haar Cascade XML file. Without this model, the program cannot detect faces. |
| Convert to grayscale | Haar Cascade mainly works on brightness and edge patterns, so grayscale is faster and more suitable than color. |
| Improve contrast | equalizeHist makes facial features clearer when lighting is uneven. |
| Reduce noise | GaussianBlur removes tiny noisy patterns that can create false detections. |
| Detect faces | detectMultiScale finds face-like regions and returns rectangles around them. |
| Filter bad boxes | The program removes very small, oddly shaped, and border-hugging detections to reduce false positives. |
| Remove duplicates | Highly overlapping rectangles are treated as duplicate detections of the same face. |
| Crop and save | Each final face rectangle is cropped from the original color image and saved into a frame-wise folder such as cropped_faces/frame_1/face_1.jpg. |